

Criteria for Project Selection

Introduction

This document defines the criteria we will use to choose projects for our software engineering course. It emphasizes time-to-value through rapid iterations, supported by CI/CD to enable frequent build, test, and delivery. Selection also considers contextual relevance, available engineering engines, scalability, and measurable evaluation criteria to ensure we build solutions that can be validated quickly and effectively.

1 Time-to-value

We **shall** design for rapid delivery, *i.e.*, deliver meaningful increments within a short time window (say weekly). And, **prioritize fast feedback over perfection**, in order to reduce waiting time between decisions and results.

Initially, you may think of it as:

- Ship a usable increment within 2 weeks.
- Complete discovery + prototype + validation in one 2-week sprint.
- Success is defined by what can be delivered in the first 2-week iteration.

1.1 CI/CD

You might have guessed that the core requirement of this project is to showcase multiple iterations of **build, test, and ship**, fast enough for the user/client to get quick feedback about the product.

To support our short weekly iteration cadence, we should automate build/test/release as much as practical. CI/CD is a good way to achieve that—starting with the lightest pipeline we can (e.g., CI first) and expanding only if it improves delivery.

2 Heuristics

2.1 1. Relevance

Contextual relevance: The problem affects our immediate stakeholders and is within our current scope to influence.

A rough equivalent of need statement. We shall prefer that your problem be of:

- **Local relevance**, i.e., relevance to our local environment, region, or team.
- **Immediate applicability**, i.e., we can act on it now.
- **Near-term impact**, i.e., its outcomes are soon visible.

2.2 2. Engine availability

This is an engineering course and not a research course. Hence, a necessary heuristic for selection should be that one or more mature engine(s) should be available as an engine for the core solution of the problem.

We shall address the issues of scalability and complexity here, rather than fundamental algorithms itself. Intuitively, we will focus on administrative aspects of software development, more than operational or creative aspects.

PS: Research delays **shall not** be entertained.

2.3 3. Scalability

With foresight, your project should be scalable in the following contexts:

1. **Scalable:** Theoretically speaking, your proposition should in principle scale to a larger audience than intended.
2. **Operationally Deployable:** Practically speaking, your proposition should be deployable using the available solutions for an extremely large audience.

2.4 4. Evaluation Criterion

Evaluation criterion: How will we know it worked?

A good proposition/solution usually includes a way to judge whether the solution actually works.

The evaluation criterion should be measurable (and ideally attributable).

Examples

- **Problem:** “Students can’t find answers fast.”

Solution: “Add an FAQ search.”

Evaluation criterion: “Search results find the correct answer within 10 seconds, and users report satisfaction $\geq 4/5$.”

- **Problem:** “The app is slow.”

Solution: “Optimize queries.”

Evaluation criterion: “Page load time decreases from 3s to under 1.5s in production.”

2.5 5. Higher-order goal

Treat this as a secondary heuristic, i.e., something to be fulfilled with a lower priority.

It can be very useful—but mostly as a framing layer, not as the whole engineering plan. A higher-order goal, e.g., address climate change, helps:

1. Guides priorities.
2. Creates measurable outcomes.
3. Makes the problem-solution link easier to justify.

But we still need to translate that “why” (climate change) into a measurable problem, e.g.:

- Reduce e-waste and “time-to-replace”.
- Reduce network energy and hardware load.
- Lower lifecycle impact by improving reliability (or durability).
- Sustainable operations.
- Digital efficiency.

3 Conclusion

In selecting a project, we prioritize measurable time-to-value through rapid, incremental delivery and fast feedback. CI/CD is the practical mechanism that makes shipping dependable. We focus on problems with contextual relevance, available engineering engines, and scalable, deployable solutions—validated through clear evaluation criteria—while keeping higher-order goals as supportive framing rather than primary drivers.